

LaTeX Indexing Editor

Design Overview: subsystems, classes and the shared core

Rewritten 5 September 2026, after Phase 6a. This document is the source; the `.rtf` and the `.pdf` beside it are generated from it by `bookindexcore/documentation/md_to_rtf.py`. Do not patch those.

1. Introduction

The LaTeX Indexing Editor is a PySide6 desktop application for building and maintaining embedded indexes in LaTeX source. It opens a folder of `.tex` files as a project, scans them for `\index` macros, and presents everything it finds in two coordinated views, a hierarchical index tree and a spreadsheet-style entry table, through which entries can be created, edited, re-filed and deleted. Every change is written back into the LaTeX source itself; the application adds a working layer over the document rather than replacing it with a database of its own.

Each project carries a SQLite database alongside its `.tex` files, holding the parsed index data and the project's own settings. That database is a cache and a coordinate map, not the source of truth: the `.tex` files are authoritative, and the application can rebuild the database from them at any time.

What changed in this edition

Half of this application is no longer in this repository. Between 9 August and 4 September 2026 its host-neutral code was extracted into a shared package, `bookindexcore`, which the Word index editor and ToA Builder also consume; the branch was merged to `main` as Phase 6a on 5 September.

The previous edition of this document was written on 5 August and describes the application as it stood before that. Measured on the day of the merge, it named **45 classes as this application's that had moved into the core**, was missing **17 that had been added here**, and mentioned the shared package **not once**. That is not ordinary drift, and it is the reason for a rewrite rather than an amendment.

To stop it recurring, the class inventory below is now checked by `probes/design_doc_drift.py`, which reports any class defined here and not named, and any class named here that has since moved into the core.

Design principles

- **Layer separation.** The code is organised as models, views and controllers, and the boundary is enforced by convention throughout: views hold no business logic and never touch persistence, models import no Qt widgets, and controllers own all coordination. **A second boundary now runs beside it**, between this application and the shared core; see section 2.
- **Signals over direct calls.** Subsystems communicate through Qt signals rather than holding references to each other, so a view can be replaced or driven from a test without the controller knowing the difference.
- **Long work goes off the UI thread.** Project loading, project-wide search and the LaTeX export pipeline each run in a worker object inside a dedicated `QThread`, following one shared worker/thread pattern. The authority-record lookup behind name inversion is the one

exception: it is short-lived and a single call deep, so it runs on a small thread pool and returns through a queued signal rather than carrying a thread of its own.

- **Deferred, transactional writes.** Index changes accumulate in memory and are written to the database in a single transaction when the project is saved, so a save is all-or-nothing.
- **Undo is a first-class model.** Every operation that mutates the index records a command describing both halves of the change, so it can be reversed across the source file, the database and the views together.

Architectural shape

One controller sits at the centre. `AppPipelineController` is constructed once at startup, is handed every model and view, and wires the signal graph that connects them. Feature-specific controllers hang off it, each owning one tool or one pane. Subsystems below are grouped by responsibility rather than by folder; models, views and controllers belonging to the same feature are described together.

Data flows in a consistent direction. A user action reaches a view, which emits a signal; a controller interprets it, asks a model to change state, and writes through to the `.tex` source via the document I/O layer; the model then signals back and the views refresh. Persistence is touched only at the ends of that chain, on project load and on save.

2. What is here, and what is shared

`bookindexcore` is a package of everything about an embedded index that is not specific to one host application. It has no dependency on this application, and this application depends on it in one direction only.

It is documented separately, in

`bookindexcore/documentation/bookindexcore_for_host_developers.md`, whose Appendix A is an index of its public API and is itself guarded by a drift probe. That document is the authority on the package's internals and this one does not repeat it: a second description of 192 classes would disagree with the first the day either moved.

What this application takes from it

- **The index model and its commands.** Entry records, the tree engine that gives a heading its identity, and the undo commands that describe both halves of a change.
- **The dialect seam.** What an `\index` macro means, how a sort key is written, what a cross-reference looks like: this application supplies the LaTeX answers and the core holds the question.
- **The presentation widgets.** The entry table, the entry window, the search panels, the theme system, the preferences shell and the dialogs common to every host.
- **Persistence.** Four of the project database's seven tables, and the whole of the machine-wide shared store; see section 5.
- **Names, sorting and authorities.** Name inversion, filing keys, declared alphabets, and the citation grammar behind the Table of Authorities tool.

What stays here

Everything that is true of LaTeX and not of Word or InDesign: the macro grammar and its parser, the source-coordinate arithmetic, the editor tabs and the syntax highlighter, the toolchain integration that runs `makeindex` or `xindy`, the project workspace over a folder of `.tex` files, and the controllers that assemble all of it into an application.

The rule the split follows: if a piece of code would say the same thing in the Word editor, it belongs in the core; if it mentions a backslash, it belongs here.

How it is installed

Both this application and the package are installed *editable* into the virtual environment, the package as a `.pth` file holding one path. Nothing is pinned: Phase 6a merged the extraction without cutting a tag, so a change crossing both repositories is still one commit in each. The pin arrives with Phase 6b, and only then does a two-repository change acquire a coordination cost.

3. Subsystems

Seventy-five classes are defined in this application. Each is named once below, with what it is for.

3.1 Application shell and orchestration

The frame the rest of the application is mounted into, and the object that assembles it. The shell classes are deliberately passive: the main window owns layout and nothing else, exposing its panels for the pipeline controller to populate. This is what allows the entire object graph to be constructed headlessly in tests exactly as `main.py` constructs it.

- **AppPipelineController** — The central orchestrator. Constructs and connects every controller, model and view, and owns the project open/save/close workflows, the recently opened project list, the undo stack, the auto-save timer, the off-thread name lookups and the exit prompts.
- **LatexIndexWindow** — The main window. Owns layout and menu assembly and nothing else, exposing its panels for the pipeline controller to fill.
- **MainMenuBar** — The menu bar, built as a passive structure whose actions are connected by the controllers that own the features behind them.
- **WorkspaceLifecycleController** — Opening, closing and reopening a project as a lifecycle, including what has to be torn down before another project can be opened in the same window.

3.2 Project and workspace management

A project is a folder of `.tex` files plus the database beside them. This subsystem resolves one, tracks which files are in indexing scope, and notices when files have changed outside the application.

- **ProjectScopeController** — Which files are in indexing scope, and the pruning that takes a file out of it without forgetting it.
- **FileTreePersistence** — The project database itself: creation, schema, and the file-scope and custom-command tables this application owns.

- **FileTreeView** — The Workspace Files tree.
- **LatexFolderFilterProxy** — Filters that tree to the files a project cares about.
- **PrunedFilesController, PrunedFilesDialog** — Manage Pruned Files: what has been taken out of scope, and putting it back.
- **ProjectLoadWorker, SafeProjectLoadThread** — The first-open scan and later reloads, off the UI thread, in the worker-and-thread pattern the export pipeline also follows.
- **ProjectSidebarView** — The sidebar hosting the workspace and index trees.

3.3 Document I/O and editor tabs

The layer that owns the bytes. Everything that reads or rewrites a `.tex` file goes through here, which is what makes the coordinate arithmetic and the checksum bookkeeping possible in one place.

- **DocumentIOController** — Reading, rewriting and checksum-stamping source files, and the distinction between a file open in a tab and one only on disk.
- **LatexTextBackend** — The dialect-facing backend: applies an edit to LaTeX source and reports what moved.
- **MacroEntry** — One macro occurrence as the backend sees it.
- **EditorTab** — A tab holding one file's text. Read-only by design; undo and redo are the only user-reachable route to a change.
- **LatexEditor** — The text widget inside a tab.
- **LatexHighlighter** — Syntax highlighting for LaTeX, including the `\index` macros this application cares about.

3.4 Index data model and persistence

The parsed index, and the coordinates that tie every entry back to the character span it came from.

- **LatexIndexParser** — Finds `\index` macros in source and parses each into an entry, including its options and its position.
- **IndexTag, NoteLocator** — One parsed macro and its parts, and a note locator within a page reference.
- **IndexEntryModel** — The in-memory index: entries, their headings and their references.
- **ReferenceCarrier** — What carries a reference's page, style and range state.
- **IndexTreeModelEngine** — Heading identity and hierarchy, and the allocation of heading ids before any row exists.
- **EntryModifierModel** — The entry table's model, and the display cache behind it.
- **_LatexCodec** — Escaping and unescaping between what an indexer types and what a macro holds.
- **LatexDialect** — This application's answers to the core's dialect questions: what a macro means, how a sort key is written, what a cross-reference looks like.

3.5 Index editing and undo

Every mutation of the index passes through here, and each records both halves of its change so it can be reversed across the source, the database and the views together.

- **IndexEditController** — Creating, editing, re-filing and deleting entries, and the journalling that defers the database write to save time.
- **IndexNavigationHelper** — Finding an entry's place in the source and taking the user there.
- **ProjectCommandManagerController**, **ProjectCommandManagerDialog** — The undo stack made visible: what has been done, and stepping back through it.

3.6 Index presentation

The two coordinated views, and the formatting rules that decide how an entry reads on screen.

- **IndexTreeView** — The hierarchical index tree.
- **CaseInsensitiveItem** — A tree item that files without regard to case.
- **SourceCoordinate** — Where a tree item points in the source.
- **IndexTextFormatterDelegate** — Draws an entry as it should read, including the label-only italic a managed cross-reference takes.
- **EntryModifierController** — The entry table: selection, editing and keeping it in step with the tree.

3.7 Cross-references

Managed cross-references are the one part of the index that cannot be rebuilt by re-scanning the source, which is why they are written to the database immediately and emitted into a generated file.

- **CrossReferenceController** — Creating and maintaining managed cross-references, and generating `cross_refs.tex`.
- **CrossReferenceList** — The cross-reference tab.
- **XrefTypeDelegate** — Choosing a cross-reference's type in that list.
- **LegacyXrefMigrationDialog** — The one-time offer to bring hand-written cross-references under management, and the recorded decision if it is declined.

3.8 Tools and analysis

Each tool is a controller and a dialog, reachable from the Tools menu, and each reports rather than transforms: nothing here changes the index without the indexer accepting it.

- **CheckIndexController**, **CheckIndexPrefs** — The Check Index rules and which of them a project runs.
- **RangeConsistencyController**, **RangeConsistencyDialog** — Malformed ranges, overlapping ranges and enclosed point references, and what applying a fix does to each.
- **BulkRepairController** — Applying one decision across many entries.
- **ToaController**, **ToaPlan**, **ToaEntry**, **ToaApplyResult**, **ToaPrefs** — The Table of Authorities tool: the plan built from the parsed citations, the entries in it, what applying it did, and the preferences that shape it.

- `HeadNoteDialog` — The index's head note.

3.9 LaTeX output and export

Compiles the real files on disk through the external toolchain, which is why it reflects the last save rather than unsaved work.

- `IndexExportController` — The export workflow and its progress.
- `RtfExportThread`, `RtfExportWorker` — Running the toolchain off the UI thread.
- `RtfExportEngine` — Turning a sorted index into RTF.
- `RtfExportMetadata` — What the generated document says about itself.
- `RtfExportView` — The export pane.
- `RtfViewerDialog` — Previewing the result without leaving the application.

3.10 Configuration and preferences

Settings live in two places on purpose: global defaults in `QSettings`, and a per-project copy seeded from them on first open, so changing a default never reaches back into a project already under way.

- `PreferencesPersistence` — Reading and writing the settings groups.
- `QSettingsGlobalStore` — The global side of that.
- `SortPrefs`, `PresentationPrefs` — The sorting and presentation groups, as this application scopes them.
- `IndexPrefsConfigModel`, `IndexPrefsData`, `IndexPrefsConfigController`, `IndexPrefsConfigDialog` — The LaTeX-specific index settings, including the generated preamble.
- `LatexCommandRegistryModel`, `CreateCommandController`, `CreateCommandDialog`, `LatexCommandWizardDialog` — The custom `\index`-bearing commands a project defines, and the wizard that writes one.
- `LatexIndexController` — Ties the index settings to the index itself.

3.11 Context menus

One manager per surface, each assembled from the core's shared base so that a right-click means the same thing in every host.

- `EditEntryContextMenuManager` — In the entry window.
- `FileTreeContextMenuManager` — In the Workspace Files tree.
- `IndexTreeContextMenuManager` — In the index tree.

4. How the subsystems fit together

Four flows account for most of the interaction between subsystems. Following them is the quickest way to understand where any given piece of behaviour lives. **Each now crosses the seam**, and the crossing is named.

Opening a project

Project management resolves the folder and database, then runs the loader on a worker thread. The parsed result is ingested by the index data subsystem and handed to index presentation to display. Configuration seeds the project's settings from the global defaults on first open and loads them from the project's database thereafter. The application shell enables project-scoped menu items, starts the auto-save timer, and records the project in the recently opened list once the load has actually succeeded. A remembered project is checked for existence only when it is chosen, rather than every time the File menu opens.

Across the seam: the parse is this application's, the model the result is poured into is the core's, and the widgets that display it are the core's.

Editing an entry

Index presentation emits the user's action. Index editing interprets it, asks document I/O to rewrite the macro span in the source, updates the index data model and shifts the coordinates of everything after it in that file, records an undo command, and refreshes the views. The database write is not performed; it is journalled.

Across the seam: the rewrite and the coordinate arithmetic are this application's, through `LatexTextBackend`; the command that records both halves of the change is the core's.

Saving

The application shell commits every open tab's buffer through document I/O, then drains both journals into the database in a single transaction via project management. It re-stamps each written file's checksum so the application's own work is not later reported as an outside change, and clears the session backups, which moves the point a Discard reverts to. Auto-save runs this same workflow on a timer.

Across the seam: the transaction covers tables created by both codebases; see section 5.

Exporting

Configuration supplies the toolchain paths and the generated preamble, project management supplies the base document and output folder, and export runs the external tools off the UI thread and renders the sorted result. Because it compiles the real files on disk, it reflects the last save rather than unsaved work.

Across the seam: the sort keys the engine files by are the core's, and the toolchain that consumes them is this application's.

Dependency direction

The layering holds in one direction. Views depend on nothing but Qt and the style authority. Models depend on nothing but each other and the persistence layer. Controllers depend on both and are the only classes that know how a feature is assembled. Where two subsystems must cooperate, they do so

through a controller and a signal rather than a direct reference, which is why the tools, the export pipeline and the editing pipeline can each be exercised in isolation.

And one direction across the seam. This application imports `bookindexcore`; the package imports nothing from here. Where the core needs a LaTeX answer it asks for one through a declared seam, and this application supplies it.

Deliberate exceptions

- Cross-references write to the database immediately, because their generated `.tex` file cannot be rebuilt by re-scanning the source.
- Project configuration writes immediately, because a setting such as the base file would be wrong not to take effect until save.
- A file with no open tab is rewritten on disk as it is edited, while a file open in a tab is changed in its buffer and reaches disk on save.

5. The databases

The application uses two SQLite databases. One is created per project and lives beside the `.tex` files it describes. The other is **machine-wide and shared with the other applications built on `bookindexcore`**, which is a change from the previous edition of this document: it used to be application-wide and to live with the installed program.

Neither holds anything the application could not do without. The project database is a cache and a coordinate map over source that remains authoritative; the shared store is a cache of decisions and declarations. Both can be deleted, and the cost is rebuild time or a re-answered question rather than lost work.

5.1 The project database

One file per project, written into the project folder as `<project name>_index_manifest.db` and found again on reopen by that suffix. It carries four kinds of state: which files are in scope, the parsed index and where every macro sits in its file, the managed cross-references, and the project's own settings.

Its purpose is to make reopening a project cheap and to give each project its own configuration. On the first open the folder is walked and every `.tex` file parsed; on later opens the file list and the index are read back from here instead, and the folder is only re-walked on an explicit resync. Because the `.tex` files remain the source of truth, nothing in this database is irreplaceable. The cost of that trade is that edits made outside the application go unnoticed until detected, which is what the recorded per-file checksums are for.

Most of it is written at save time. Index changes accumulate in memory as journalled operations and are drained into `project_headings` and `project_references` in a single transaction, so a save is all-or-nothing. Two things deliberately break that rule and write immediately: `project_cross_references`, because the `cross_refs.tex` file generated from it cannot be rebuilt by re-scanning the source; and `project_metadata`, because a setting such as the base file would be wrong not to take effect until the next save.

The schema is seven tables, and since the extraction it is created by two codebases. That is worth stating plainly, because a reader looking for `CREATE TABLE project_headings` in this repository will not find it.

table	created by	holds
<code>project_files</code>	this application, <code>models/file_tree_persistence.py</code>	every file the loader has seen, and whether it is in indexing scope
<code>project_file_sync_state</code>	this application	per-file checksums, so an outside edit can be detected
<code>project_custom_commands</code>	this application	the custom <code>\index</code> -bearing commands a project defines
<code>project_metadata</code>	<code>bookindexcore</code> , <code>persistence/index_repository.py</code>	the project's own settings, as a key/value store so a new preference needs no schema change
<code>project_headings</code>	<code>bookindexcore</code>	one row per distinct heading path; a heading's identity is its text and depth together
<code>project_references</code>	<code>bookindexcore</code>	one row per occurrence, pointing at its heading
<code>project_cross_references</code>	<code>bookindexcore</code>	the managed cross-references, written immediately

Heading ids are allocated in memory by the tree engine rather than by SQLite, so a heading can exist and be referenced before its row has been written.

The two schemas are versioned independently. This application's migrations are numbered from 1.0.0 and the core's from 2.0.0, and each keeps its own version key rather than sharing one, so a host can gain a table without claiming to have moved the core's schema.

The three tables this application owns

Field by field, from `models/file_tree_persistence.py`. For the four the core creates, see `bookindexcore_for_host_developers.md`, which documents them and is checked against the code by its own probe.

`project_files` — every file the loader has seen in the project folder, and whether it counts as being in indexing scope. This is what allows the Workspace Files tree to be rebuilt from the database rather than by walking the folder again on every open.

field	purpose
<code>absolute_path</code>	Full path to the file. Primary key, and the identity every other part of the application refers to the file by.
<code>file_name</code>	The bare file name, for display in the tree.
<code>is_active</code>	1 if the file is in indexing scope, 0 if pruned. Manage Pruned Files toggles this rather than deleting the row, so a pruned file can be restored.
<code>last_indexed</code>	When the row was last written.

project_file_sync_state — written whenever the index is known to match a file's current content, which is a fresh scan, a manual resync, or an auto-heal after an external edit. Compared against each file's live checksum on project load, which is how drift accumulated while the application was not running is detected.

field	purpose
<code>file_path</code>	The file this checksum is for. Primary key.
<code>checksum</code>	The content hash as of the last known-good sync.
<code>synced_at</code>	When that was.

project_custom_commands — the custom `\index`-bearing commands added to this project from the global registry. It stores an independent name-and-body snapshot taken when the command was added, **deliberately decoupled from the global entry it was copied from**, so editing the global registry does not silently change how an existing project parses.

field	purpose
<code>name</code>	The command name, without the backslash. Primary key.
<code>body</code>	The command's definition as this project holds it.
<code>added_at</code>	When it was added to the project.

5.2 The shared store

```
%LOCALAPPDATA%\DH Indexing\shared\indexing.db
```

One file for the machine, not one per application. It was moved there on 4 September 2026 at the indexer's direction, on the ground that a name decision made in one application should not have to be made again in another. It is owned by `bookindexcore.store`; this application reaches it through the package and never opens it directly.

Five tables:

table	holds
cache	name decisions: what an authority record said, and what the indexer chose
house_profiles	authored house styles for the Table of Authorities
alphabets	declared alphabets an indexer has written
model_tiers	which language models have been assessed, and how they scored
schema_version	the store's own version, for migration

The old per-application folder is kept and holds a pointer to the new location, because an installed build reads that pointer and nothing else; without it such a build would create a fresh empty database and the indexer's decisions would look lost.

6. Keeping this document true

`probes/design_doc_drift.py` compares this document against the code:

```
.venv\Scripts\python.exe probes\design_doc_drift.py
```

It reports any class defined in this application and not named here, and any class named here that has since moved into `bookindexcore`. It does not judge the descriptions, which are prose.

Run it before regenerating the RTF. The previous edition of this document went four weeks without that check and acquired 45 wrong entries and 17 omissions, which is what the probe exists to make impossible rather than merely unlikely.